Hazard Analysis
Lake

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

Table 1: Revision History

Date	**Developer(s)**	**Change**
October 14, 2026	All	Initial revision

Contents

1 Introduction **1**
- 1.1 Definition of a Hazard . . . 1

2 Scope and Purpose of Hazard Analysis **1**
- 2.1 Purpose . . . 1
- 2.2 Scope . . . 1
- 2.3 Losses . . . 2

3 System Boundaries and Components **2**
- 3.1 Coordinator . . . 2
- 3.2 Model Partitioner . . . 2
- 3.3 Inference Engine . . . 3
- 3.4 Tensor I/O . . . 3
- 3.5 Boundaries . . . 3

4 Critical Assumptions **3**

5 Failure Mode and Effect Analysis **4**

6 Safety and Security Requirements **8**

7 Roadmap **9**
- 7.1 Within the capstone timeline . . . 9
- 7.2 After the capstone . . . 10

8 Appendix — Generative AI Use Disclosure **12**
- 8.1 AI Tools Used . . . 12
- 8.2 How and Where Used . . . 12
- 8.3 Verification of AI-Generated Content . . . 12

9 Appendix — Reflection **13**

List of Tables

1 Revision History . . . i
3 Safety and security requirements scheduled against project milestones . . . 9

List of Figures

1 Introduction

This document is the hazard analysis for Lake. Lake splits a large language model (LLM) into blocks of layers and runs each block on a different device on a local network, so that the devices can run a model together that none of them could run alone. The leader node's coordinator assigns blocks of layers to the devices and accepts prompts from the user. Each follower node runs its block and passes the result on to the next device.

1.1 Definition of a Hazard

For this document, a hazard is a state or condition of Lake that, combined with conditions in its environment that the system does not control, can lead to one of the losses listed in Section 2.3. Examples of such conditions are a device losing power, a congested network, or an untrusted device on the same network. For example, a device being assigned more layers than its memory can hold is a hazard, and the crashes that follow are the loss. A failure mode is a specific way a component can fail, and it can create a hazard.

Lake has almost no potential for physical harm, so its hazards are mainly about privacy, security, the correctness and availability of responses, and the effect of the system on the user's own devices.

2 Scope and Purpose of Hazard Analysis

2.1 Purpose

The purpose of this analysis is to find the ways Lake can cause a loss while the project is still in the requirements phase, when a hazard can be removed by changing a requirement instead of the code. Each hazard found in the FMEA leads to a safety or security requirement, which is added to the SRS and then verified through the V&V Plan.

2.2 Scope

In scope:

- The software components of Lake: the coordinator, the model partitioner, the inference engine on each device, the communication between devices, and the user interface.
- The interaction of these components with the user's devices, the local network that connects them, and the model files that are loaded.

Out of scope:

- The content the model generates. Harmful, biased or false text produced by a correctly running model is a property of the model, which the user chooses and Lake treats as an input. Incorrect output caused by Lake itself is in scope.
- Defects in the operating system, drivers or GPU hardware. These are outside Lake's control, so this analysis does not try to prevent them. It does cover their effects, such as a device crashing partway through a request, because Lake must respond to them.
- Attackers outside the local network, and physical access to a device. Lake is not exposed to the internet, and a person with physical access to a device already controls everything on it.

2.3 Losses

The hazards in this document can lead to the losses below.

ID	Loss	Description
L1	Loss of privacy or security	Prompts, responses or data passed between devices are seen by a person or device that is not meant to see them, or an untrusted device gives an attacker access to data or code on the user's devices.
L2	Loss of output integrity	The user receives a wrong or incoherent response because of Lake, for example from data corrupted between devices or a wrong assignment of layers, without knowing that it is wrong.
L3	Loss of availability	A request fails or never completes, or a session is lost partway through generating a response.
L4	Harm to the user's devices or work	Lake uses so much memory, processing time or power on a device that the user's other applications crash, unsaved work is lost, or the device itself crashes or freezes.

3 System Boundaries and Components

3.1 Coordinator

This component exists on all nodes, but the following responsibilities are only run on the leader node:

- Providing an OpenAI-API compatible API to the user for use with tools like OpenCode.
- Tracking cluster health, and detecting node failures.
- Notifying nodes of partitioning changes.
- Rebuilding the cluster and saving in-progress requests in the case of node failure, removal, or addition.
- Measuring node connection performance.

On a follower node, it is primarily responsible for managing the inference engine on its device, following the instructions of the leader node.

3.2 Model Partitioner

This component determines how to partition the model among the participating devices. It decides at which points to cut the graph, how large each partition is, and which device gets each partition. This component is integral to ensuring that each device is assigned an appropriate partition given the device's computational and networking performance.

3.3 Inference Engine

This component is the high-performance inference engine running on each device. It will handle execution of the given model partition for each device. It will call into ggml using the Rust-to-C FFI (TODO: glossary) to control the execution of a device's model partition.

3.4 Tensor I/O

This component handles the compressing, sending, receiving, and decompressing of tensors between devices. It will leverage gRPC for high-performance communication, with the potential to use a custom solution should gRPC fail to meet our requirements in practice.

3.5 Boundaries

The system boundaries for Lake primarily consist of:

- The Coordinator's OpenAI-API compatible API.
- Model files (GGUF) loaded from disk.
- Networking components between devices (Thunderbolt, Ethernet, Wi-Fi, routers).
- The OS and GPU resources on each individual device.

4 Critical Assumptions

We assume:

- The model file (GGUF) provided to Lake is already a functional LLM before partitioning.
- User requests are only made through the API on the leader device.
- Only one model will be run by the pool at a time.
- The local area network connecting the devices is private, and trusted by the user.

5 Failure Mode and Effect Analysis

The following Failure Mode and Effect Analysis (FMEA) table summarizes potential failure modes for the Lake system based on each component. It also details potential causes and effects for each of the failures, along with a recommended action to mitigate the potential failure. Components are listed based on their appearance above in Section 3. Beyond this, there are also hazards listed for general communication failures that may take place in between nodes or components, as well as privacy failures.

Component	Failure Modes	Effects of Failure	Causes of Failure	Recommended Action	SR	Reference
Coordinator	Incorrect or outdated device health statistics are reported back to the user.	Coordinator sends requests to device that is unable to handle requests and inference fails.	Health checks are too infrequent from the coordinator or user interface displays older data than available.	Scheduled health checks to each device and removal of data from user interface as soon as new data reaches leader node.	SR0-1	H0-1
	Coordinator component in leader node fails to notify follower node of partitioning change.	Follower node sends data to other failed follower node and inference fails.	Lack of proper and timely notification system and health checks from leader node.	Scheduled health checks to each device prior to each message to ensure inference is not lost on a failed node.	SR0-2	H0-2
	Single device node is allocated 100% of the model.	Fails the core purpose of Lake and overworks single device by avoiding parallelization.	Improper boundaries on model partition algorithm.	Boundaries to ensure each device is allocated >0% of the model, and rebalancing checks made throughout a session to verify boundary is not violated.	SR1-1	H1-1
	The model is insufficiently partitioned and cannot be split up amongst the device pool. (# Partitions < # Devices)	Model runs on only part of available hardware and does not achieve best possible performance.	Poor constraints on model partition algorithm.	Ensure that layers are split up into enough pieces to span all devices (pigeonhole principle) and build checks into initial algorithm to guarantee each device handles part of inference.	SR1-2	H1-2

Model Partition						
	Device is allocated a partition larger than it's available memory.	Device crashes and loss of network.	Improper consideration of device hardware requirements and lack of tolerances.	Ensure tolerances built into partition algorithm to keep models at (100-t)% of device availability (t is specified threshold that will be determined in testing).	SR1-3	H1-3
Inference Engine	Follower device fails or crashes during local inference.	Inference is lost and response cannot be returned to user.	A myriad of local failures could cause the device to not complete the inference. This could be hardware constraints or other device processes interfering.	Leader device is able to prevent failures from being shown to user and send retry request(s) to produce proper output.	SR2-1	H2-1
	Node receives incoherent tensors and cannot produce output to user.	User does not receive a response to their prompt and cannot use the system.	Communication failure (lost or corrupted data) between two nodes.	Place checks throughout the inference process for corrupted or lost tensors and restart inference from leader, temporarily removing the node where corruption/loss occurred from the network. Additionally, checksums can be implemented to catch network corruption, with the system restarting if the checksums do not match.	SR3-1	H3-1

Tensor I/O	Leader node fails to extract compressed tensors.	User does not receive a response to their prompt and cannot use the system.	*i* - Follower device fails to produce proper output or *ii* - Leader cannot interpret output and run final inference to produce output.	Using the checks identified in H3-1, identify the node causing the failure and temporarily remove it from the network before restarting. If no problem is indicated from any follower node, restart inference and notify user of the issue.	SR3-2	H3-2
	Communication between two nodes times out between devices or fails entirely.	Failure of system, loss of inference if midway through session.	Failure of communication protocol between nodes at hardware level (Thunderbolt, Wi-Fi, Ethernet).	Monitor average communication times between nodes and alert user to check connections if it exceeds a threshold. This can be measured through packet loss and throughput metrics. Restart inference if communication is lost and remove failed node.	SR4-1	H4-1
	Device health status communication fails.	Leader node unaware if device fails, loss of system is possible if unknown device's performance is degraded.	Failure on behalf of leader node to send outbound requests, or failure of follower nodes to receive and respond to requests.	Monitor for devices not returning health checks, after multiple missed, remove device from inference network until health response is received by leader. In the case of leader node failure, the system's follower nodes will interpret a lack of incoming health requests as leader failure and shut down since no further coordination can take place.	SR4-2	H4-2

General Communication	Communication becomes noticeably slower for user compared to expected performance.	Users leaving the system and opting for another provider.	Inefficiency in between nodes or complete network slowness. Model is split up across too many devices causing too much communication overhead for each prompt.	Simplification of network. If inference is taking too long to complete based on historical system data, remove least capable devices on the network to reduce communication overhead.	SR4-3	H4-3
Privacy	Untrusted node on the device network.	Allows unintended access to the system and potential loss of data to bad actors. Reduces user trust in the system and may lead to loss of users.	Lack of initial authorization of each node or operation of the system on an untrusted network where unknown devices can join.	Ensure that system is only operated on limited networks where all potential devices are known. Alternatively, the leader node must gain approval from the user before adding an unknown device to the network.	SR5-1	H5-1
	Data leakage from a node leading to untrusted access or data leakage.	Untrusted users who have access to any of the participating nodes can see and steal data. Reduces user trust in the system and could lead to degraded or failed performance.	Lack of initial authentication, or storage of unnecessary temporary data on intermediary devices.	Ensure that any devices in the network are explicitly approved by the user and that any unnecessary data is not stored on intermediary devices.	SR5-2	H5-2

6 Safety and Security Requirements

- **SR0-1** – The system shall identify and discard stale device health data from determining device availability by configuring a time-to-live for valid data.
- **SR0-2** – The system shall only route inference requests to nodes that currently have a confirmed model layer partition.
- **SR1-1** – The system shall assign at least two nodes for model layer partitioning during distributed inference.
- **SR1-2** – The system shall ensure all model layers are assigned to a node, and that each device configured to run model layers receives a non-zero model layer partition assignment.
- **SR1-3** – The system shall reject models and layer allocations that are estimated to exceed the memory capacities of the Lake network or individual nodes, including a configured safety margin. Related SRS requirements: FR.S.2.1, FR.S.2.2.
- **SR2-1** – The system shall attempt an automatic bounded recovery and retry on inference requests if a follower node fails during inference. If the follower node is unable to be recovered within the configured recovery limits, the system must report to the API that the request has failed. Related SRS requirements: FR.S.1.10, FR.S.1.11, FR.S.1.13.
- **SR3-1** – The system shall identify corrupted and incomplete tensors throughout the distributed inference with integrity checks and communication timeouts to discard affected data and retry requests.
- **SR3-2** – The system shall log failed tensor decoding metadata and report a failure to the coordinator so the inference request can recover, subject to SR5-2.
- **SR4-1** – The system shall configure a timeout for communication between nodes, and handle exceeded timeouts according to the appropriate recovery or failure policy.
- **SR4-2** – The system shall mark a node as unavailable for inference or model layer partitioning when the node health criteria are not met, and require a successful health check before marking the same node as available. During a leader node failure, follower nodes shall detect and initiate a shutdown. Related SRS requirements: FR.S.1.24 and FR.S.1.25.
- **SR4-3** – The system shall utilize configured values or historical data to determine a baseline inter-node communication latency and notify the coordinator when that baseline is exceeded by a specified tolerance.
- **SR5-1** – The system shall require user approval on the leader node to authorize a follower node, and user confirmation on the follower node to confirm the authorization before it can participate in the Lake network. Related SRS requirements: FR.S.1.17, FR.S.1.18, FR.S.1.20, FR.S.1.21, NFR.S.12.
- **SR5-2** – The system shall not record prompts, responses, tensor data, and KV cache content in diagnostic logs. Related SRS requirements: NFR.S.10.

7 Roadmap

Every safety and security requirement in Section 6 is tied to a hazard in the FMEA, and each is scheduled below against the project milestones from the Development Plan. The order follows one rule: a requirement is implemented in the same phase as the capability whose failure it guards. Without that capability the hazard cannot occur, and with it the hazard is live, so the guard cannot wait for a later phase. Requirements that guard a capability outside the capstone scope are deferred, and the reason is given.

7.1 Within the capstone timeline

Table 3 gives the phase, the target date, and the reason for each requirement. The dates are the internal deadlines for each milestone.

Table 3: Safety and security requirements scheduled against project milestones

Req.	Phase	Target	Why this phase
SR1-1	Proof of concept	Nov 23, 2026	The PoC is a model split across two processes, then two devices. A split that puts everything on one node is not a PoC at all, so the check is part of the first partitioner.
SR1-2	Proof of concept	Nov 23, 2026	Same reason: every layer assigned, every node given a non-empty block, is the definition of a correct split and is tested from the first run.
SR1-3	Proof of concept	Nov 23, 2026	The PoC splits a model by hand across two devices and the first way it fails is a block larger than the device's memory. The memory check goes in with the first loader.
SR5-2	Proof of concept	Nov 23, 2026	The PoC writes the first diagnostic logs, and the cheapest time to keep prompts, responses, tensors, and KV cache content out of them is before any log line exists. Every later log (SR3-2 included) inherits the rule.
SR0-2	Revision 0	Feb 1, 2027	Dynamic allocation lands in January. Once blocks can move between nodes, the coordinator can route a request to a node whose block is not yet confirmed, so this guard ships with reallocation.
SR0-1	Revision 0	Feb 1, 2027	Health data only matters once nodes can join and leave mid-session. The time-to-live on health reports is built with the health-check loop that reallocation depends on.
SR4-2	Revision 0	Feb 1, 2027	Marking a node unavailable until it passes a health check is the other half of the same loop.
SR4-1	Revision 0	Feb 1, 2027	Timeouts between nodes are how a departed node is detected. Without them reallocation never triggers.

Table 3 continued

Req.	Phase	Target	Why this phase
SR2-1	Revision 0	Feb 1, 2027	Bounded retry after a follower fails is the user-facing behaviour of reallocation, and it is what the Rev 0 demo shows.
SR5-1	Revision 0	Feb 1, 2027	Node agents joining and leaving are a Core system deliverable. The PoC has no join flow (the two devices are configured by hand), so the first time an unknown device can ask to join is Rev 0, and the approval on both sides ships with that flow.
SR3-1	Final demonstration	Mar 15, 2027	Integrity checks on tensors between nodes belong to the integration and communication testing task, which runs after reallocation is stable. Until then a corrupted tensor surfaces as a failed request, which SR2-1 already handles.
SR3-2	Final demonstration	Mar 15, 2027	Logging failed decodings is added with SR3-1, since it is the record that SR3-1's checks fired.
SR4-3	Final demonstration	Mar 15, 2027	A latency baseline needs enough measured runs to set a tolerance. The per-token profiler feeds it, and the user interface that shows the warning is a Rev 0 deliverable, so the alert is wired up during final integration.

The thresholds in these requirements (the memory safety margin in SR1-3, the time-to-live in SR0-1, the timeouts in SR4-1, and the latency tolerance in SR4-3) are set to conservative values first and tuned from measurements on the team's own machines before each demonstration. The values and how they are verified are recorded in the V&V Plan.

7.2 After the capstone

The following are known hazards or mitigations that the capstone timeline does not cover. Each is listed with the condition under which it would move into scope.

- **Encryption between nodes.** The critical assumption in Section 4 is that the local network is private and trusted, which is also the Problem Statement's stretch goal. Hazard H5 in the FMEA covers an untrusted node and data left on intermediary devices, and SR5-1 and SR5-2 handle both by user approval and by keeping request data out of logs. Neither stops a device that is already on a hostile network from reading or injecting tensors in transit. A potential user we interviewed for the SRS, who runs an on-premises assistant on export-controlled data, named encrypted traffic and company-managed machines as conditions for deploying anything like Lake. If the stretch goal is taken on, the assumption is removed, that hazard gets an FMEA row, and encryption and cryptographic node authentication become requirements.
- **Backing off when a device owner is working.** Loss L4 covers a device that becomes unusable because of Lake. The current requirements limit memory use (SR1-3) but do not pause or throttle inference when the owner is active. The interviewee's first condition for pooling office machines was that Lake back off the moment the owner starts working. This

becomes a requirement once Lake is run on machines that are not dedicated to it, which is after the capstone.

- **No telemetry, and auditability.** The same interview asked that Lake send nothing outside the cluster, including usage or crash reports, and that its behaviour be auditable. The SRS privacy requirement (NFR.G.4.1) states that no request data leaves the cluster and that Lake sends no telemetry; it is verified by a network capture during a session, and SR5-2 keeps request data out of the logs that stay on the devices. Auditability has no requirement yet and is deferred with the encryption work above, since it matters for the same organisational users.

8 Appendix — Generative AI Use Disclosure

8.1 AI Tools Used

Anthropic Claude (Sonnet 5.5, Opus 5.5, and Fable 5.1 through Claude Code), OpenAI GPT (GPT-6.1 Sol), Alibaba Qwen (3.8 Flash), and Google Gemini (3.8 Flash, falling back to 3.5 Flash Lite, through the repository's automated rubric-review action) were used in conjunction with team member discussions while writing this document.

8.2 How and Where Used

Claude, GPT, and Qwen were used to brainstorm and review sections with the team, for the LaTeX formatting of the FMEA table in Section 5 (a landscape table with fixed column widths that spans several pages), and for a general pass over the document for spelling and grammar. The hazards, causes, effects, and recommended actions in the table were written by the team.

Claude was used to ideate the Roadmap (Section 7), this disclosure, and the reflection appendix from the team's own FMEA, safety requirements, Development Plan schedule, and the SRS user interview, and to check that the hazard and requirement identifiers used in the roadmap match those in Sections 5 and 6. It did not identify hazards or write safety requirements.

The Gemini rubric review provides advisory comments on each pull request; the team reads them and decides what, if anything, to change.

8.3 Verification of AI-Generated Content

Proposed changes to the document are reviewed through pull requests, and the branch rules require at least two approvals from team members before merging. The roadmap's requirement identifiers were checked against Section 6, its hazard identifiers against the FMEA, and its dates against the schedule in the Development Plan and the SRS project chapter. The reflection draft was edited by the team so that every statement about what we did is one a team member confirms, and the FMEA table was compared against the author's hazard list to confirm that the formatting step changed no content. The team is responsible for the accuracy, completeness, consistency, and appropriateness of this document, regardless of whether material was generated by a person or an AI tool.

9 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?
4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?

---

1. Splitting the document by component worked. Matt had already named the five components for the SRS, so the system boundary section and the FMEA used the same names, and each hazard ID carries its component number (H1 for the partitioner, H3 for Tensor I/O). That made it easy to see which component had no hazards yet and go looking for them, and it means the safety requirements trace to the SRS without a separate mapping table. The losses table in the scope section also helped: once the four losses were written down, deciding whether something was a hazard became a matter of asking which loss it led to.
2. The main pain point was that the Hazard Analysis and the SRS were written in the same two weeks by different people, and they depend on each other in both directions. The FMEA's recommended actions become the SRS safety requirements, but the FMEA also needed the SRS component list and the critical assumptions to exist first. We handled it by writing the SRS component section first and reusing its names and numbering here, and by opening one pull request per section so that a section could be reviewed as soon as it existed rather than waiting for the whole document. The second pain point was the FMEA table itself: a seven-column table that has to break across landscape pages is awkward in LaTeX. Ethan used an AI tool for the table formatting only, which is recorded in the disclosure.
3. Two of the risks were on paper before this deliverable. The Development Plan's proof of concept section names network overhead making the split slower than a single device (H4-3), and the Problem Statement's dynamic allocation goal is built around devices dropping out of the pool mid-session (H2-1). A block larger than a device's memory (H1-3) was implicit

in how the Development Plan describes the partitioner, but had not been written down as a failure. The rest came out of the FMEA. Walking the coordinator's responsibilities one at a time produced H0-1 and H0-2, stale health data and a missed partition notification, which we had not thought of because we had been picturing the happy path where every message arrives. The partitioner rows H1-1 and H1-2 came from asking what a correct partition even is, which forced us to write down that every layer is assigned exactly once and every node gets a non-empty block; that definition is now also the formalization in the SRS. The Tensor I/O rows H3-1 and H3-2 came from the question of what happens when a message arrives but is wrong rather than missing, which is a different failure from H4-1 and needs a different check. The privacy rows H5-1 and H5-2 were added while the FMEA was under review. The SRS already said that untrusted devices must not join a cluster and that request data is not saved, but neither had been written down as a hazard, and a review comment moved H5-2 from node approval, which H5-1 already covers, to data left on devices that are already in the pool. Finally, the user interview for the SRS, which happened during this deliverable, added two items to the roadmap that no one on the team had listed: backing off when a device owner is working, and sending no telemetry.

4. Two other types of risk matter for Lake. The first is privacy and security risk: a software product can leak data without anything breaking. Lake's whole reason to exist is that prompts and documents stay on the user's own machines, and losses L1 and L2 in the scope section are about data being seen or altered, not about anything crashing. If that guarantee fails quietly, the product is worse than the cloud service it replaces, because the user believes they are protected. The second is risk to the user's other work and property: Lake runs on computers that people use for other things, and a process that takes all the memory or the GPU can lose someone's unsaved work or make a machine unusable, which is loss L4. This kind of risk is easy to ignore because it does not show up in Lake's own tests, but it decides whether anyone lets Lake onto their machine at all. Both types also carry a reputational risk for the team: a tool that leaks data or crashes laptops is not one anyone recommends.